

COSE436 Interactive Visualization (Fall 2025)
Instructor: Prof. Won-Ki Jeong
Due date: Dec 21, 2025, 11:59 pm.

Assignment 4: Isosurface Volume Rendering

In this assignment, you will implement direct volume rendering. Specifically, you are required to implement a ray casting algorithm using isosurface rendering. An example of isosurface volume rendering result is as follows:



Figure 1. Isosurface volume rendering example

1. Volume Viewer (40 points)

First, you need to implement an OpenGL viewer with virtual trackball interaction (mouse-based rotation and zoom). Ensure that the left mouse button controls rotation and the right mouse button controls zoom. You may choose to manipulate either the volume itself or the camera position and viewing direction — both approaches are acceptable

2. Ray casting (60 points)

To make the volume renderer working, you need to complete the fragment shader. There are two parts you need to implement as follows:

2.1 Ray-cube intersection

Once the eye position and per-pixel location are passed to the fragment shader,

then the viewing ray can be generated. Then, you need to compute intersection between the viewing ray and six sides of the cube to find two endpoints, one is the *entry* and the other is the *exit* point.

2.2 Ray casting

Once the entry and exit points are found, then we need to walk on the viewing ray, starting from the entry point to the exit point (see Fig 2). The number of samples (or the step dt) can be defined on your own. At each sample location, you can directly access 3D texture.

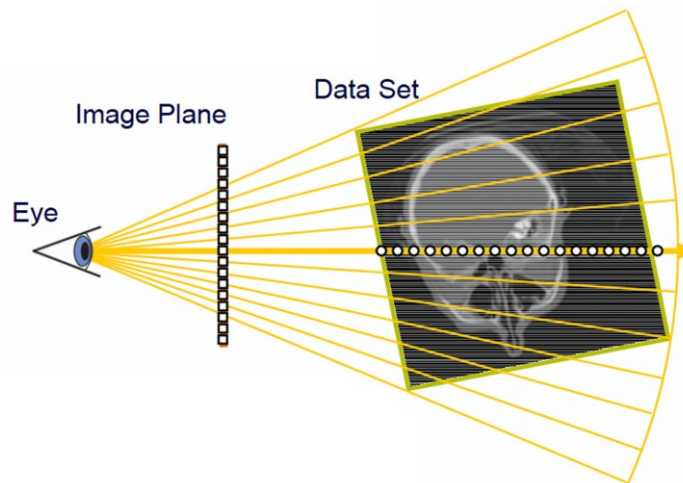


Figure 2. Ray casting

For iso-surface rendering, first you should find the location where zero-crossing occurs. This can be detected as comparing two consecutive sampling points (say x_1 and x_2) and check whether the given iso-value is between these two sample values. Once you detected the zero-crossing, you may need to refine the location (e.g., starting from x_1 but reduce the step size by half and find a zero-crossing location, repeat this until you find a sufficiently small range of (x_1, x_2) that includes the iso-value). Then, you should compute the surface normal at the zero-crossing point by calculating gradient vector (dx, dy, dz) . Last, you should compute the color on the sampling point using the Phong illumination model (diffuse, specular, ambient). More details can be found in the lecture notes (Lecture 23).

You also need to implement keyboard callback to interactively change isovalue using cursor keys.

Up key: Increasing isovalue

Down key: Decreasing isovalue

What to submit:

- Main.cpp and shader (.vert and .frag) files

Good luck!!!